

RAP 後端開發練習 3：Behavior Definition 基礎——Managed 與 Unmanaged 對照

Lecture

為什麼這一課要教兩種寫法

rap01 介紹過 Managed / Unmanaged 的概念差異，也提過本課程只教 Managed (Unmanaged 需要紮實的 OOP 底子，份量太大)。但開發過程中意外發現一個對這系統很關鍵的事實：這套系統的 RAP **Managed Runtime** (CUD 寫入部分) 被 SAP 官方標記為「尚未對外釋出」，任何 Managed BDEF 的 **create/update/delete** 操作，執行到底層都會直接 Dump (詳見下方 Part C)。所以這一課調整成：

- **Part A：Managed 語法**——當作知識儲備，跟官方教材 / 未來的 ABAP Cloud RAP 課程接軌，語法在這系統上能寫、能啟用，但沒辦法真正執行
- **Part B：Unmanaged 語法**——這系統上真正能跑、能驗證的版本，份量比 rap01 原本規劃的重一些，但既然 Managed 執行不了，這是這系統唯一能讓你看到「資料真的寫進去了」的路
- **Part C：兩者的差異總表 + 為什麼這系統只有 Unmanaged 能跑**

Part A：Managed BDEF 語法 (知識儲備，這系統無法執行)

Managed 的意思是：CRUD 的實際資料庫存取 (**INSERT/UPDATE/DELETE**) 由框架自動產生，你只需要用宣告式語法告訴框架「這個實體支援哪些操作」。在看完整範例之前，先逐段認識會用到的語法元素：

- **managed;**：BDEF header 只有這一個字，宣告這個實體用 Managed 模式 (跟 Part B 的 Unmanaged header 寫法完全不同)。本系統不支援 **strict(2)** 這種新式語法 (rap01 已提過)。
- **define behavior for <CDS View 名稱> alias <別名>**：綁定到某個 CDS Interface View，**alias <別名>** 幫這個實體取一個簡短的別名，之後 EML 呼叫時就用這個別名，不用打完整的 CDS View 名稱。
- **persistent table <表格名稱>**：告訴框架資料實際存在哪張表。如果 CDS View 欄位名稱跟表格欄位名稱完全一致，不需要額外的 **mapping for** 區塊；名稱不一致時才需要另外寫 **mapping for <表格> { ... }** 明確對應每個欄位。
- **lock master**：這個實體是鎖定管理的「主體」，框架自動處理 **ENQUEUE/DEQUEUE**，不用自己寫鎖定邏輯。
- **etag <欄位>**：ETag (樂觀鎖定用的版本標記) 指定用哪個欄位當版本依據——注意這系統不支援官方教材常見的 **etag master <欄位>** 寫法，只能寫單純的 **etag <欄位>** (不帶 **master**)，下方會解釋原因。
- **{ create; update; delete; }**：宣告這個實體支援哪些標準操作，沒宣告的操作 (例如只寫 **create**；不寫 **delete**；) 就代表這個實體不允許該操作。
- **field (mandatory) <欄位>**：把這個欄位標記成必填，**create** 時沒填會被框架擋下來。
- **field (readonly) <欄位1>, <欄位2>**：這些欄位使用者不能直接透過 **create/update** 設值，通常搭配 rap05 的 Determination 自動填值使用。

認識完各個語法片段後，這是這一課完整的 Managed BDEF 範例 (綁定 rap02 建好的 **ZI_RAP02_TASK**)：

```
managed;
```

```

define behavior for ZI_RAP02_TASK alias Task
persistent table zrap02_task
etag created_at
lock master
{
    create;
    update;
    delete;

    field ( mandatory ) description;
    field ( readonly ) created_at, created_by;
}

```

⚠ 這一課踩到的語法坑：**etag master** 在這系統不支援。官方教材 / 新式 ABAP Cloud RAP 常見寫法是 **etag master created_at** (帶 **master** 關鍵字)，這系統啟用直接報 `"authorization | draft | late | { | ~" expected, not "created_at"`。——錯誤訊息列出的合法後續 token 完全沒有 **etag**，代表這系統文法不認得 **etag master** 這個組合。回頭比對 rap01 提過的系統既有標準物件 (**SCR_E_DBDEV / A_ProductionSupplyArea**)，兩者都是寫單純 **etag** <欄位>，完全沒有 **master**——這是這系統適用的語法 (見 `.claude/rules/sap-adt-mcp.md` 第 40.11 節)。

Managed 版本的 EML 語法 (**ZR_RAP03_DEMO**，見答案區) 也已經寫好、語法正確、成功啟用，可以當範例讀，但**不要期待它能真正執行成功**——原因見 Part C。

Part B：Unmanaged BDEF——這系統上真正能跑的版本

Unmanaged 的意思是：CRUD 的實際邏輯你自己寫在一個 **ABAP 類別**裡，框架只負責派發呼叫，不管你怎麼存資料。這一課用一個簡化的獨立範例示範 (**ZRAP03_UMTEST / ZI_RAP03_UMTEST**，只有 **id / descr** 兩個欄位，刻意比 rap02 的 **Task** 簡單，聚焦在 Managed vs Unmanaged 的機制差異，不重複設計資料模型)。

先看 **BDEF 語法**跟 **Part A** 的兩個關鍵差異 (看完差異再看完整範例，比較容易記住為什麼要這樣寫)：

- **header** 是 **implementation unmanaged in class <類別名> unique;**，不是單純 **managed;**——因為所有邏輯都要靠這個類別實作，BDEF 必須指名是哪個類別。
- 完全沒有 **persistent table** 子句——Managed 需要這個子句告訴框架「資料存在哪張表，讓框架自動生成 SQL」；Unmanaged 不需要，因為資料庫存取全部由你自己在實作類別裡寫，框架不插手。

這是 **ZI_RAP03_UMTEST** 的完整 Unmanaged BDEF：

```

implementation unmanaged in class zbp_i_rap03_um4 unique;

define behavior for ZI_RAP03_UMTEST alias Test
lock master
{
    create;
}

```

實作類別分成兩個檔案，先認識各自的角色跟語法元素，再看完整程式碼：

- **Global 類別本體**：只是一個空殼，用 `FOR BEHAVIOR OF <CDS View>` 宣告「這個類別是誰的 Behavior 實作」，真正的邏輯不寫在這裡。
- **Local Types Include (真正的 Handler 邏輯)**：
 - `INHERITING FROM cl_abap_behavior_handler`：所有 Unmanaged Handler 類別的必要基底類別，提供 `failed/reported/mapped` 這些保護屬性給你在方法裡直接用（不用自己宣告）。
 - `FOR LOCK / FOR MODIFY / FOR READ`：方法用途分類，`IMPORTING ... FOR CREATE <alias>` 這種寫法把方法綁定到 BDEF 宣告的特定操作。
 - **LOCK 方法必須實作**（就算是空的）——即使沒有真的要做什麼鎖定邏輯，框架也要求這個方法存在。
 - **READ 方法也建議實作**——即使 BDEF 的 `{}` 裡沒有明寫 `read`，Unmanaged 實體通常還是需要基礎的讀取能力（EML / OData 內部會用到）。

這不是一般 OOP 的 `METHODS` 語法：**Handler Method** 宣告怎麼跟一般方法不一樣

先直接對照：一般 OOP 方法宣告，參數型別一定要用 `TYPE` 明講：

```
METHODS lock
  IMPORTING it_lock TYPE some_table_type.
```

這一課用的寫法：

```
METHODS lock FOR LOCK
  IMPORTING it_lock FOR LOCK test.
```

兩處不一樣（查證官方 ABAP 語言文件 `ABAPHANDLER_METH_LOCK / ABAPHANDLER_METH_MODIFY / ABAPHANDLER_METH_READ` 確認，不是憑印象寫的）——⚠ 這整套語法只在一個地方合法：**Local Class** 要繼承 `cl_abap_behavior_handler`、且宣告在某個 **ABAP Behavior Pool** 的 **Local Types Include** 裡；脫離這個情境，`FOR LOCK / FOR CREATE` 這種寫法在一般的 **Class** 裡直接是語法錯誤：

1. 方法名稱後面緊接的 `FOR LOCK / FOR MODIFY / FOR READ`：這是「Handler Method 類別」宣告，是固定關鍵字之一（`LOCK / MODIFY / READ`，還有 Determination / Validation / Action 專用的幾種），告訴編譯器「這個方法要扮演哪一種 RAP 角色」——一般 OOP 方法完全沒有這種東西。
 - ⚠ 容易搞混的地方：`create` 這個方法用的類別關鍵字是 `FOR MODIFY`，不是 `FOR CREATE`——`MODIFY` 是涵蓋 `CREATE / UPDATE / DELETE / Action` 的統稱類別，`CREATE` 只出現在後面第二個 `FOR` 子句裡（下一點），是完全不同位置的兩個關鍵字，不要以為 `create` 方法應該對應 `FOR CREATE`。
2. 參數後面的 `FOR LOCK test / FOR CREATE test / FOR READ test`（取代一般的 `TYPE <型別>`）：這是最不像一般 OOP 的地方——一般方法參數一定要接 `TYPE` 明講型別，這裡完全沒有 `TYPE`，換成 `FOR <操作> <BDEF別名>`。`test` 對應的正是 BDEF 裡 `alias Test` 這個別名，意思是「這個參數的型別，去 `ZI_RAP03_UMTEST` 目前生效的 Behavior Definition 裡自動算出來，不用你自己寫」——這個機制官方叫 **BDEF derived type**（衍生型別），編譯器實際推導出的型別分別是：

| 你寫的簡寫 | 編譯器實際推導出的型別 | `FOR LOCK test` | `TYPE TABLE FOR KEY OF <Test 對應的 BDEF 實體>` | `FOR CREATE test` | `TYPE TABLE FOR CREATE <Test 對應的 BDEF 實體>` | `FOR READ test` (`IMPORTING` 那個參數) | `TYPE TABLE FOR READ IMPORT <Test 對應的 BDEF 實體>` | `RESULT et_result` (`READ` 方法的輸出參數) | `TYPE TABLE FOR READ RESULT <Test 對應的 BDEF 實體>` |

FOR <操作> <別名> 裡的 <操作> 到底是什麼：這是第二個 FOR 子句自己的關鍵字，用來指定「這個參數對應 BDEF { } 裡宣告過的哪一個具體操作」——跟第 1 點的 **<類別>** (**LOCK / MODIFY / READ**) 是兩層不同的東西，不要混為一談：**<類別>** 是「這個方法扮演哪一種 RAP 角色」的粗分類，**<操作>** 是「這個方法底下的這個參數，具體對到 BDEF 宣告過的哪一項操作」。合法值要看方法屬於哪個 **<類別>** (查證官方語法文件 **ABAPHANDLER_METH_LOCK / MODIFY / READ** 逐一列出的完整清單)：

| 方法的 **<類別>** | **<操作>** 合法值 | 對應 BDEF { } 裡宣告的內容 | ---|---|---| | **FOR LOCK** | **LOCK <bdef>** (永遠只有這一種) | **lock master / lock dependent by** 這類鎖定宣告 | | **FOR MODIFY** | **CREATE <bdef>** | { **create;** } | | **FOR MODIFY** | **CREATE <bdef>~<association>** (Create-by-Association) | **association _Booking { create;** } 這種掛在關聯上的 **create** | | **FOR MODIFY** | **UPDATE <bdef>** | { **update;** } | | **FOR MODIFY** | **DELETE <bdef>** | { **delete;** } | | **FOR MODIFY** | **ACTION <bdef>~<action名稱>** | BDEF 裡自訂的 **action <名稱>**; (rap07 會教) | | **FOR READ** | **READ <bdef>** | 讀取這個實體本身 (這一課的 **read** 方法用的就是這個) | | **FOR READ** | **READ <bdef>~<association>** (Read-by-Association) | 透過關聯讀取子實體 (例如從 Travel 讀 Booking) | | **FOR READ** | **FUNCTION <bdef>~<function名稱>** | BDEF 裡自訂的 **function <名稱>**; (進階功能，本課程用不到) |

這一課的三個方法對應到：**lock** 是 **FOR LOCK test** (**<操作>** 固定是 **LOCK**)、**create** 是 **FOR CREATE test** (**<操作>** 是 **CREATE**)、**read** 是 **FOR READ test** (**<操作>** 是 **READ**) ——**<操作>** 用的關鍵字剛好都跟方法名稱一樣，容易誤以為是巧合，其實是因為這幾個方法的名稱是我們自己取的 (**meth** 可以自由命名)，特意取跟操作一致的名字方便閱讀，語法上完全可以取別的名字 (例如把 **create** 改叫 **handle_new_task**，只要 **FOR CREATE test** 這個綁定沒改，功能完全一樣)。

1. **RESULT et_result** (**READ** 方法特有)：另一個一般 OOP 沒有的關鍵字，用來額外宣告一個輸出參數——RAP Handler Method 不能用 **RETURNING**，只能用這種專屬語法；**et_result** 一樣不寫 **TYPE**，型別照樣是編譯器從 BDEF 自動推導。
2. **failed / reported / mapped** 完全不用宣告就能直接用：官方文件明講這三個是「隱含可用」的 **CHANGING** 參數——就算像這一課的 **lcl_handler** 一樣，方法簽章裡完全沒寫 **CHANGING failed reported**，**METHOD create. ... ENDMETHOD.** 的方法本體裡照樣可以直接寫 **failed-test = ... / reported-test = ...**，編譯器會自動幫你這幾個變數準備好。一般 OOP 裡沒有這種「憑空冒出來能用的變數」，方法能用的變數不是自己的區域變數、就是簽章上明確宣告的參數。

一句話總結：**METHODS <名稱> FOR <類別> IMPORTING <參數> FOR <操作> <BDEF別名> [RESULT <輸出參數>]** 整組語法，是 ABAP 專門為 RAP Handler Method 開的宣告式綁定語法——**METHODS / IMPORTING / RESULT** 這些關鍵字看起來眼熟，但 **FOR <類別>** (方法名稱後) 跟 **FOR <操作> <別名>** (參數名稱後) 這兩處，是一般 OOP 完全沒有的 RAP 專屬擴充，作用是讓編譯器直接從 BDEF 反推出正確的參數型別、同時把這個方法綁定到 BDEF 宣告的某個操作。這也呼應前面那題「全域類別跟 Local Class 之間靠什麼連結」的答案：連結不是你手動接的，是這整套宣告式語法讓編譯器 / **RAP Runtime Engine** 自動接起來的。

這段寫法是照抄這系統既有標準物件 **CL_SD_BEHV_SALESORDERMANAGE** (**C_SalesOrderManage** 的實作類別，rap01 提過的標準 Unmanaged 範例) 讀出來的真實模式，不是憑空猜的：

```
" Global 類別本體 ( ZBP_I_RAP03_UM4 )
CLASS zbp_i_rap03_um4 DEFINITION
    PUBLIC
    ABSTRACT
    FINAL
    FOR BEHAVIOR OF zi_rap03_umtest.
```

```

PUBLIC SECTION.
PROTECTED SECTION.
PRIVATE SECTION.

ENDCLASS.

CLASS zbp_i_rap03_um4 IMPLEMENTATION.
ENDCLASS.

```

```

" Local Types Include ( 真正的 Handler 邏輯 )
CLASS lcl_handler DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
METHODS lock FOR LOCK
IMPORTING it_lock FOR LOCK test.

METHODS create FOR MODIFY
IMPORTING it_create FOR CREATE test.

METHODS read FOR READ
IMPORTING it_read FOR READ test RESULT et_result.
ENDCLASS.

CLASS lcl_handler IMPLEMENTATION.

METHOD lock.
ENDMETHOD.

METHOD create.
LOOP AT it_create INTO DATA(ls_create).
INSERT zrap03_umtest FROM @( VALUE #( client = sy-mandt id = ls_create-id descr
= ls_create-descr ) ).
ENDLOOP.
ENDMETHOD.

METHOD read.
LOOP AT it_read INTO DATA(ls_key).
SELECT SINGLE id, descr FROM zrap03_umtest WHERE id = @ls_key-id INTO
@DATA(ls_data).
IF sy-subrc = 0.
APPEND VALUE #( %key = ls_key-%key id = ls_data-id descr = ls_data-descr ) TO
et_result.
ENDIF.
ENDLOOP.
ENDMETHOD.

```



```
ENDCLASS.
```

全域類別 (`ZBP_I_RAP03_UM4`) 跟 Local Class (`lcl_handler`) 之間，靠什麼連結起來？

看這兩段程式碼會發現一個奇怪的地方：`ZBP_I_RAP03_UM4` 的 `IMPLEMENTATION` 區塊是完全空的，`lcl_handler` 也沒有任何屬性、方法去參照或建立 `ZBP_I_RAP03_UM4` 的實例——兩者的原始碼裡找不到任何一行明確的關聯（沒有 `TYPE REF TO`、沒有繼承、沒有 `CREATE OBJECT`）。BDEF 也只提到 `ZBP_I_RAP03_UM4` 這一個類別名稱（`implementation unmanaged in class zbp_i_rap03_um4 unique;`），完全沒提過 `lcl_handler`。那 RAP 框架究竟怎麼知道「真正的邏輯要去 `lcl_handler` 裡找」？

答案是：這個連結不是靠程式碼裡任何一行「參照」建立的，是靠三個結構性規則同時成立，由 **ABAP 編譯器與 RAP Runtime Engine** 在背後自動解析出來的（查證官方 ABAP 語言文件 `ABENABP_CL_ABAP_BEH_HANDLER` / `ABENABP_HANDLER_CLASS_GLOSRY` 確認）：

1. **物理位置**：`lcl_handler` 必須寫在 `ZBP_I_RAP03_UM4` 這個全域類別自己的 **Local Types Include**（官方技術名稱叫 **CCIMP include**，是這個全域類別本身的一部分，不是獨立物件）裡面——這是唯一的「連結」，純粹靠「住在同一個類別裡」，沒有任何顯式的參照語法。
2. **FOR BEHAVIOR OF `zi_rap03_umtest`**：全域類別 `ZBP_I_RAP03_UM4` 這個宣告，把「這整個類別（包含它 CCIMP include 裡的所有 Local Class）」跟「`ZI_RAP03_UMTEST` 的 Behavior Definition」綁在一起——框架知道要去這個類別（含它包起來的所有 Local Class）裡找實作，就是靠這一行。
3. **INHERITING FROM `cl_abap_behavior_handler` + 方法簽章的 `FOR <操作> ... FOR <別名>` 語法**：`lcl_handler` 繼承 `cl_abap_behavior_handler` 標記自己是一個「Handler Class」；`METHODS create FOR MODIFY IMPORTING it_create FOR CREATE test` 這句裡的 `FOR CREATE test`，`test` 對應的正是 BDEF 裡 `alias Test` 那個別名——編譯器會拿這個別名去比對 `ZI_RAP03_UMTEST` 目前生效的 Behavior Definition，確認「喔，這個方法要處理的就是 BDEF 裡宣告的 `create;` 這個操作」。三個條件同時成立，RAP Runtime Engine 才會認得這個方法。

更關鍵的一點：官方文件明講 Handler Class 隱含是 **ABSTRACT** 跟 **FINAL**，原因是「只有 **RAP Runtime Engine** 才能建立實例、呼叫它的方法」——你自己完全不能（也不需要）寫 `CREATE OBJECT lo_handler TYPE lcl_handler` 這種程式碼，`lcl_handler` 從語法上就禁止你這樣做。這也解釋了為什麼你在 `ZR_RAP03_UMTEST` 這種驗證程式裡完全看不到 `lcl_handler` 這個名字——你的程式只會用 EML（`MODIFY ENTITIES OF zi_rap03_umtest ENTITY Test CREATE ...`）跟 CDS View / BDEF 打交道，中間「該呼叫哪個類別的哪個方法」全部是 RAP Runtime Engine 在背後幫你解析、實例化、呼叫的，開發者完全不用（也無法）手動介入這一段。

跟一般 **OOP**「物件之間互相持有參照」的直覺不一樣：一般寫 Class 之間的協作，通常會看到 `DATA go_xxx TYPE REF TO yyy` 這種明確的參照鍵；RAP 的 Handler Class 綁定走的是「宣告式 + 命名規則」的模式（有點像 ABAP Unit 測試類別怎麼被 `SE80 / ABAP Unit` 框架自動找到並執行的邏輯——你也不會在程式裡手動 `CREATE OBJECT` 一個測試類別再呼叫它的方法，是框架自動掃描、實例化、執行），這是 RAP 框架設計上刻意的選擇：開發者只管把邏輯寫在正確位置、簽章寫對，其餘的「怎麼串起來」交給框架，減少手動接線容易出錯的地方。

⚠ 這一課實測踩到的兩個坑：

1. `VALUE #( mandt = ... )` 寫錯——這張表的 Client 欄位自己取名叫 `client`（不是內建保留字 `mandt`），`VALUE` 建構子要對應欄位實際名稱，寫 `mandt =` 會啟用失敗，這只是單純欄位名稱對應錯誤，不是 RAP 語法限制。

2. **READ** 方法結果表的技术鍵欄位是 **%key**，不是 **%tky**——啟用時系統直接在錯誤訊息裡建議了正確名稱 (**No component exists with the name "%TKY", but there is a component with a similar name "%KEY"**) 。

什麼是 EML

前面用 **lcl_handler** 建好了「怎麼處理 CREATE」的邏輯，這裡要看「呼叫端怎麼觸發它」——也就是 **ZR_RAP03_UMTEST** 這支驗證程式用到的 **EML (ABAP Entity Manipulation Language)**。查證官方 ABAP 語言文件確認 (不是憑印象寫的)：

- 官方定義：EML 是 **ABAP** 語言的一個子集 (不是獨立的語言，是內建在 ABAP 編譯器裡的一組專屬語法)，專門用來存取 **RAP BO (RAP Business Object)** 跟 **RAP BO Interface**。EML 陳述式可以讀取 / 修改 RAP BO 的「**Transactional Buffer**」(交易緩衝區，這是這一課前面已經提過的概念：**MODIFY ENTITIES** 改的資料先進這個記憶體暫存區，**COMMIT ENTITIES** 才真的觸發持久化寫入資料庫)，也能觸發把緩衝區的異動真正存進資料庫。
- 可以在任何 **ABAP** 程式裡使用：不只是這一課寫的獨立驗證程式 (**ZR_RAP03_UMTEST**) 可以用，任何 ABAP 程式、Class 的方法、甚至另一個 **RAP BO** 自己的 **Handler Method** 裡都可以用 EML 去呼叫別的 RAP BO (後者官方文件特別提到有一些專屬變體，這門課不深入)。
- 為什麼需要一種「專屬語法」，不能直接用 **Open SQL** 存取底層表？——這是 EML 存在的核心理由，也是它跟 Open SQL 最大的差異：
 - **Open SQL (SELECT / INSERT / UPDATE / DELETE)** 直接對資料庫表 / CDS View 操作，完全繞過 **BDEF** 宣告的業務邏輯——不會觸發 Lock、不會走 Determination / Validation (rap05 / rap06 會教)、也不會經過你在 **lcl_handler** 寫的任何檢查。
 - **EML (MODIFY ENTITIES)** 操作的是 **RAP BO**，執行的時候會觸發 **RAP Runtime Framework**——對 Unmanaged BO (或 Managed BO 裡的 Unmanaged 部分) 來說，就是去呼叫你寫在 ABAP Behavior Pool 裡的 Handler Method (呼應前面「全域類別跟 Local Class 怎麼連結」那題的答案：**EML** 陳述式的執行，正是觸發那整套宣告式綁定機制的起點)；對 Managed BO 來說，則是由框架自動生成的邏輯處理。也就是說，**EML** 才是「走完整 **RAP BO** 業務邏輯」的正規存取管道，**Open SQL** 是繞過這一切、直接碰資料庫的後門。
 - 這也是為什麼這一課 **ZR_RAP03_UMTEST** 裡，寫入資料用 EML (**MODIFY ENTITIES**)，但最後驗證資料有沒有真的進資料庫卻改用 Open SQL **SELECT** (前面已經解釋過原因：EML 的 **READ** 可能讀到還沒同步的緩衝區資料)——兩種存取方式各有用途，不是誰取代誰：EML 負責「正確地」修改資料 (會經過所有業務邏輯檢查)，Open SQL 適合單純查詢、或這種「繞過 RAP 框架直接問資料庫最終狀態」的驗證情境。
- **EML** 底下有一整組陳述式，這一課只用到其中兩個 (**MODIFY ENTITIES / COMMIT ENTITIES**)，完整家族還有：

陳述式	作用	這一課有沒有用到
MODIFY ENTITY / MODIFY ENTITIES	CUD 操作 (CREATE / UPDATE / DELETE / Action)	✓ 用到
READ ENTITY / READ ENTITIES	讀取 RAP BO 資料 (含 Read-by-Association)	沒用到 (這一課驗證改用 Open SQL，見上一點)
COMMIT ENTITIES	觸發 RAP Save Sequence，真正寫入資料庫	✓ 用到
ROLLBACK ENTITIES	放棄 Transactional Buffer 裡還沒 Commit 的異動	沒用到
GET PERMISSIONS	查詢目前使用者對某個 RAP BO 操作的權限 (rap 課程權限機制不在範圍內)	沒用到
SET LOCKS	明確鎖定 RAP BO 實例	沒用到 (lock 方法由框架自動觸發，這一課不用自己呼叫)

MODIFY ENTITY / READ ENTITY (單數) 是只操作一個實體的簡化寫法 (Short Form)，**MODIFY ENTITIES / READ ENTITIES** (複數，這一課用的) 是可以一次處理多個實體、多種操作的完整寫法 (Long Form)——這

門課統一用複數的完整寫法，跟官方教材 / Cheat Sheet 的主流慣例一致。

EML 語法：MODIFY ENTITIES / FAILED / REPORTED / COMMIT ENTITIES 逐段解說

認識完 EML 是什麼之後，回到 ZR_RAP03_UMTEST 實際用到的語法。同樣先認識語法元素，再看完整程式碼（查證官方 ABAP EML 語言文件與 Cheat Sheet 確認，不是憑印象寫的）：

- **MODIFY ENTITIES OF <CDS View> ENTITY <alias> <操作> FIELDS (...) WITH VALUE #((...)) ...**：呼叫 CUD 操作的固定格式：
 - **OF <CDS View>**：指定要操作哪個 RAP BO（CDS Interface View 名稱）。
 - **ENTITY <alias>**：指定要操作哪個實體，用 BDEF 裡宣告的 **alias**（這一課是 **Test**），不用打完整的 CDS View 名稱。
 - **<操作>**：**CREATE** / **UPDATE** / **DELETE** 三選一（還有其他進階操作，這一課只用到 **CREATE**）。
 - **FIELDS (...)**：列出這次操作要傳的欄位清單。
 - **WITH VALUE #((...))**：**CREATE** / **UPDATE** 用 **WITH**，**DELETE** 用 **FROM**（rap03 學習目標已經提過這條規則）；裡面是一個內部表的 **VALUE** 建構子，每一列（用 **(...)** 包起來）對應一筆要新增/修改的資料，可以一次傳多筆。
- **%cid**：Content ID 的縮寫，是這一系列資料在這次呼叫裡的臨時識別碼，由開發者自己取名（字串，例如 'C1'）——不是資料庫欄位，作用是讓框架能把「你傳進去的這一系列」跟「之後 **MAPPED** 回應裡查到的真正主鍵」對應起來。這一課的 **id** 是自己指定的值（不是系統自動產生），實務上用不太到 **%cid** 去反查主鍵，但語法上 **CREATE** 操作固定要求每一列都給一個 **%cid**，是 EML 的固定寫法，不能省略。
- **FAILED DATA(ls_failed)**：宣告一個變數接收「這次操作裡哪些列失敗了」——是一個依 BDEF 裡宣告的 Entity 別名分好類的巢狀結構（這一課只有 **Test** 一個實體，所以要看 **ls_failed-test**），沒有任何列失敗的話 **ls_failed-test** 是空的內部表。
- **REPORTED DATA(ls_reported)**：宣告一個變數接收「這次操作產生的所有訊息」（不分成功失敗，錯誤 / 警告 / 資訊訊息都在這裡），格式也是依 Entity 別名分類的巢狀結構——實務上這是「顯示給使用者看的錯誤訊息」的來源，例如 Fiori Elements 畫面跳出的錯誤提示，訊息文字就是從這裡取的。
- （這一課沒用到，但正式的三個回應參數是一組，要知道它存在）**MAPPED**：宣告一個變數接收「每個 **%cid** 對應到的真正主鍵值」——主鍵是系統自動產生時（例如 **UUID**、**Managed** 的 **numbering:managed**）特別重要，因為呼叫端下 **CREATE** 當下還不知道真正主鍵是什麼，要等 **MAPPED** 回應才查得到。**FAILED** / **MAPPED** / **REPORTED** 三個回應參數都是選擇性的（**[]** 括起來，可以只寫用得到的），這支程式因為 **id** 是自己指定的值，用不到 **MAPPED**，所以沒宣告它。
- **COMMIT ENTITIES**：EML 專屬的提交語句，觸發「RAP Save Sequence」（框架內部呼叫 **Saver** 方法，真正把 Transactional Buffer 裡緩衝的變更寫進資料庫的整個流程）——在這之前，**MODIFY ENTITIES** 做的變更只存在記憶體裡的暫存緩衝區，還沒真的進資料庫，一定要有這一行資料才會真的寫入。**COMMIT ENTITIES** 內部已經包含了 **COMMIT WORK**，不用（也不應該）自己再額外下一次 **COMMIT WORK** 去提交同一筆 RAP 異動——這點呼應 Enhancement 課程學過的「BAdI Implementation 不能自己下 **COMMIT WORK**」（同一個道理的不同情境：那邊是「不能」，這裡是「已經內建了，不用重複」）。呼叫完後可以檢查 **sy-subrc** 判斷有沒有成功。

認識完語法元素，這是完整的 ZR_RAP03_UMTEST：

```
REPORT zr_rap03_umtest.

WRITE: / 'before EML'.

DELETE FROM zrap03_umtest WHERE id = 'UM0001'.
```



```

COMMIT WORK.

MODIFY ENTITIES OF zi_rap03_umtest
  ENTITY Test
    CREATE FIELDS ( id descr )
    WITH VALUE #( ( %cid = 'C1' id = 'UM0001' descr = 'Unmanaged Test' ) )
    FAILED DATA(ls_failed)
    REPORTED DATA(ls_reported).

WRITE: / 'after EML, failed is initial:', xsdbool( ls_failed-test IS INITIAL ).

COMMIT ENTITIES.

WRITE: / 'after commit entities'.

SELECT SINGLE id, descr FROM zrap03_umtest WHERE id = 'UM0001' INTO @DATA(ls_check).
IF sy-subrc = 0.
  WRITE: / 'DB check OK, descr =', ls_check-descr.
ELSE.
  WRITE: / 'DB check FAILED, no record found'.
ENDIF.

```

幾個容易忽略的細節：

- 開頭 **DELETE FROM zrap03_umtest WHERE id = 'UM0001'. COMMIT WORK.**——這兩行是一般 **Open SQL**，不是 EML，純粹是測試前先清掉可能殘留的舊資料，讓每次重跑都是乾淨狀態；用的是 **COMMIT WORK**（不是 **COMMIT ENTITIES**），因為 **DELETE** 是傳統 Open SQL 語句，走傳統資料庫交易提交機制，跟後面 EML 用的 **COMMIT ENTITIES** 是兩條不同的提交路徑，不要搞混、也不能互相取代。
- 拿到 **FAILED** 之後馬上用 **xsdbool(ls_failed-test IS INITIAL)** 檢查有沒有內容——這是實務上判斷「這次 EML 呼叫到底成功了沒」最直接的方法，比等到 **COMMIT ENTITIES** 才發現失敗更早知道。
- 最後驗證資料有沒有寫進去，改用一般 **Open SQL SELECT**，不是再用一次 EML 的 **READ**——這是刻意的設計：EML 的 **READ** 讀的可能還是 Transactional Buffer 裡的資料（在還沒 **COMMIT ENTITIES** 之前，不一定跟資料庫同步），直接下 Open SQL **SELECT** 才是「完全繞過 RAP 框架、直接問資料庫這筆資料到底進去了沒」最沒有爭議的驗證方式，適合拿來做這種端對端測試的最終確認。

EML 可以像一般 ABAP 一樣 Debug 嗎？

可以——前面「什麼是 EML」已經確認 EML 是「**ABAP 語言的子集**」，也就是說 **MODIFY ENTITIES / COMMIT ENTITIES** 這些陳述式本身就是編譯進你的程式（**ZR_RAP03_UMTEST**）裡的普通 ABAP 原始碼，不是什麼外部指令碼或另一套執行環境——所以在這行程式碼上點兩下設中斷點（**Breakpoint**）、用 SAP GUI 或 Eclipse ABAP Debugger 執行到這裡，跟對 **WRITE / SELECT** 這些一般陳述式設中斷點沒有任何差別，這部分不用懷疑。

但 EML 真正特別的地方在於：它觸發的是 **RAP Runtime Framework** 這一整層框架邏輯（前面「什麼是 EML」提過），中斷點停在 **MODIFY ENTITIES** 這一行之後，如果你想繼續「單步執行」（**Step Into**）追進去，會先走過框架內部的分派機制，不一定能一路清楚地跟到你自已寫的 **lcl_handler=>create** 方法本體

——這一段框架內部調度的除錯體感，官方語言文件沒有明確描述，這裡不確定的部分不硬講。實務上更常用、也更直接的兩種除錯方式：

1. 直接在 **Handler Method** 本體裡設中斷點——例如把中斷點設在 `lcl_handler` 的 **METHOD create** 裡面（例如 `INSERT zrap03_umtest FROM ...` 這一行）。因為這也是普通 ABAP 原始碼，不管這個方法是被 EML（這一課的情境）、還是被 OData / Fiori Elements 觸發呼叫的，中斷點都一樣會被命中——這是 RAP 開發時最常用的除錯手法，直接跳過「EML 怎麼分派」這個中間過程，命中你真正關心的業務邏輯。
2. **CREATE OBJECT ... FOR TESTING**——RAP 專屬的 EML 相關語法，讓你能繞過 **Handler Class** 的 **ABSTRACT / FINAL** 限制：前面提過 Handler Class（如 `lcl_handler`）隱含是 **ABSTRACT + FINAL**，正常情況下你完全不能自己 **CREATE OBJECT**，只有 RAP Runtime Engine 才能實例化它。但官方語言文件記載了一個專門給 ABAP Unit 測試用的例外：`CREATE OBJECT class_under_test FOR TESTING`。——這個 **FOR TESTING** 附加詞會特別放行，讓你在測試類別裡直接建立 Handler Class 的實例，接著就能直接呼叫它的方法（傳入自己準備的假資料）、檢查 **failed / reported** 的結果，完全不需要透過 **EML / RAP Runtime Framework** 那一整層分派。這是官方建議的 RAP Handler Method 單元測試寫法，也是「完全隔離、只測你自己寫的邏輯」最乾淨的除錯 / 測試手段——之後如果這門課教到 ABAP Unit 測試（或你自己想試），會用到這個語法。

✅ 驗證結果（`programrun` 無頭執行，完全成功）：

```
before EML
after EML, failed is initial: X
after commit entities
DB check OK, descr = Unmanaged Test
```

資料真的寫進資料庫了，而且是無頭執行成功的——這點也意外推翻了先前記錄的「EML 沒辦法用 `programrun` 無頭驗證」（見 Part C 的完整解釋）。

Eclipse ADT 建立 Behavior Definition：Step by Step

前面看的都是 BDEF 的最終原始碼，這裡補上「怎麼從零在 Eclipse 建出這個物件的空殼」（查證 SAP 官方 openSAP《Building Apps with RAP》課程教材的實際操作步驟，不是憑空猜的；這系統畢竟是 **Classic RAP**，精靈的機械操作步驟應該通用，但精靈生成的骨架內容一律要照 **Part A / B** 教的規則改寫，例如骨架可能提示 `etag master` <欄位> 這種新式寫法，記得改成不帶 `master` 的版本）：

1. 對著要綁定的 CDS Interface View（例如 `ZI_RAP02_TASK`）按右鍵 → 選單裡直接有 **New Behavior Definition**（跟這系統一貫的模式一樣，常用 RAP 物件類型都內建成右鍵直接捷徑，不用繞 **Other ABAP Repository Object**）。
2. 跳出 **New Behavior Definition** 精靈：
 - **Project / Package / Root Entity**：自動帶出，不用填。
 - **Name**：固定跟 **CDS View** 同名，欄位是灰的不能改——這跟 Metadata Extension（名稱可以自訂，只是慣例上取一樣）是不同的規則，BDEF 的名稱規則更嚴格。
 - **Description**：系統會提議一個，可以自己調整。
 - **Implementation Type**：下拉選單選 **Managed** 或 **Unmanaged**——這就是 Part A / Part B 分岔的地方，選哪一個決定精靈接下來生成的骨架長什麼樣。
 - 按 **Next**。
3. 選傳輸請求（`$TMP` 套件直接 **Finish**，不用選）。

4. 精靈生成的骨架，是一份帶了大量註解掉的範例行 (`//persistent table <???`、`//lock master`、`//etag master <field_name>` 這種) 的檔案——這是官方精靈的固定套路：先把所有可能用到的子句都列出來但整段註解掉，你自己決定要 `uncomment` 哪些、刪掉哪些不要的、把 `<???` / `<field_name>` 這種佔位符改成真正的值，不是要你從空白檔案自己從頭打。
- 如果選 **Unmanaged**，骨架的 header 那一行會自動幫你寫好一個推測的類別名稱 (格式通常是 `zbp_i_<CDS View 名稱去掉 ZI_ 前綴>` 這種慣例)，例如 `implementation unmanaged in class zbp_i_rap03_umtest unique;`——可以直接接受，也可以自己改成想要的名稱。
5. 把骨架逐行改成 Part A / Part B 教的正確內容，存檔 (**Ctrl+S**) + 啟用 (**Activate**)。
6. **Unmanaged 專屬**：用 **Ctrl+1** 快速鍵一鍵生成實作類別骨架，不用自己手打方法簽章——BDEF 成功啟用後，把游標點在 header 那一行的類別名稱上 (例如 `zbp_i_rap03_umtest`)，按 **Ctrl+1**，會跳出快速修正選單，選 **Create behavior implementation class <類別名>** 並雙擊套用 → 選傳輸請求 → **Finish**。系統會自動生成一個繼承 `cl_abap_behavior_handler`、帶正確方法簽章的 Local Handler Class 骨架 (依 BDEF 的 `{}` 裡宣告了哪些操作，決定要生成哪些方法的空殼，例如宣告了 `create;` 就會生成 `METHODS create FOR MODIFY IMPORTING ...` 的空方法)——這一步比自己手打方法簽章可靠很多，容易漏打或打錯型別。方法本體 (`METHOD create. ... ENDMETHOD.` 裡面的邏輯) 還是要照 Part B 教的寫法自己填。

練習：自己建一個 Behavior Definition

輪到你了，分兩個難度：

- ① 必做 (**Managed**，純語法練習)：對 rap02 練習 2 建的 `ZI_RAP02_FLIGHT_PRACTICE` (那個用 `SPFLI/SCARR` 做的唯讀航班參考資料 View) 用上面的精靈步驟建一個 **Managed** BDEF，宣告 `create;/update;/delete;` (純語法練習，不用真的能執行——這個系統 **Managed** CUD 本來就一律 **Dump**，剛好也保護了你不會不小心真的把測試資料寫進標準表 `SPFLI` 裡，這個搭配是刻意設計的安全練習組合)。驗收依據：`checkruns / sap_inactive_objects` 確認啟用成功即可，不用 (也不能) 真的執行 CUD。
- ② 選做 (**Unmanaged**，進階挑戰，真的能跑)：如果想體驗完整的「BDEF + 實作類別 + 真的寫入成功」，可以自己另外建一張簡單的小表 (可以照抄 `ZRAP03_UMTEST` 的欄位設計，`id / descr` 兩個欄位就夠) + CDS View，然後幫它建一個 **Unmanaged** BDEF，用 **Ctrl+1** 生成實作類別骨架，照抄 Part B 的 `create / read / lock` 方法邏輯改寫成適合你這張表的版本，最後用 EML 或 Eclipse Data Preview 驗證資料真的寫進去。這一題份量比較重，量力而為，不強制。

完成後跟我說一下你選了哪個難度、建立過程跟最終狀態 (啟用成功與否、有沒有遇到跟 Part A / B 提過的坑一樣的錯誤訊息)，我會幫你核對。

Part C：Managed vs Unmanaged 差異總表，以及為什麼這系統只有 Unmanaged 能跑

	Managed	Unmanaged
CRUD 邏輯寫在哪	框架自動生成，你只宣告 <code>create/update/delete</code>	你自己寫在實作類別的 <code>LOCK/CREATE/UPDATE/DELETE/READ</code> 方法裡
BDEF header	<code>managed;</code>	<code>implementation unmanaged in class <類別> unique;</code>
需不需要 <code>persistent table</code>	需要	不需要 (資料庫存取自己處理)

	Managed	Unmanaged
需不需要額外的實作類別	不需要 (除非要加 Determination / Validation / Action)	一定需要
程式碼份量	少	多 (含 OOP、SQL 語句都要自己寫)
這系統能不能真正執行 CUD	✗ (見下方原因)	✓ 已驗證成功
適合的情境	全新設計的資料物件 (官方主推方向)	包一層 RAP 介面在既有系統/邏輯之上 (這系統的標準物件 <code>C_SalesOrderManage</code> 就是這樣)

為什麼這系統的 **Managed CUD** 一律失敗：使用者在 SAP GUI 執行 Managed BDEF 的 EML CREATE，得到 Runtime Error (`MESSAGE_TYPE_X_TEXT`，元件 `BC-ESI-RAP-CSP`)，根本原因是 `CL_CSP_MD_METADATA_FACTORY` 裡一段檢查邏輯——這個 CDS Entity 所在的套件，必須在一份硬編碼白名單裡 (`SBOI_RAP_CSP_TST%` 套件前綴、或幾個 SAP 內部套件/元件)，不在清單裡 (我們的 `$TMP` 或任何自訂套件都不在) 就用致命訊息擋下來，程式碼裡甚至留了開發者自己寫的英文註解：「csp isn't released for public usage until now」。查證 SAP Community 找到一篇 **2019-11** 一字不差的同款回報，時間點跟這套系統的 **S/4HANA 1909** 高度吻合——這是 **1909 版本 Managed Runtime** 尚未對客戶開放寫入功能的已知限制，不是我們哪裡設定錯了。這個檢查只卡在真正寫入的時候，純讀取 (如 Fiori Elements Preview 顯示清單) 不受影響。

⚠ 這個白名單有沒有辦法讓我們自己的套件加進去？——結論是「技術上碰不到、就算碰得到也不建議」，這不是一個該去繞過的設定項。白名單的兩組條件分別是：

- `SBOI_RAP_CSP_TST%` (套件前綴) / `/BOBF/RAP_MIGRATION` / `/BOBF/RAP_MIG_ADMINISTRATOR` (完整套件名稱) ——這幾個套件名稱本身就落在 **SAP 保留的命名空間** (`SBOI_*` 不帶 `Z/Y` 前綴、`/BOBF/` 是加了 Namespace Prefix 的正式保留命名空間)，客戶端系統的套件依規定不能取這種名字，這條路從命名規則上就走不通，不是「找不到入口」而是「這個入口本來就不對客戶開放」。
- 另一組條件是套件的 **Application Component** (套件屬性裡的「應用元件」分類) 要落在 `BC-SRV-NWD-XBR` / `BC-DWB-DIC` 這兩個 SAP 標準元件底下——雖然套件維護畫面 (SE21) 技術上可能讓你把自己套件的 Application Component 欄位改選成這兩個既有的 SAP 標準值，但這麼做等於刻意把一個客戶自訂套件偽裝成 **SAP 內部技術元件的分類**，去騙過一個 **SAP 明確標記「尚未對外釋出」的功能閘門**——這不是修正設定錯誤，是繞過供應商刻意設下的成熟度 / 授權管控，行為上跟破解軟體的授權檢查是同一類事，不建議這樣做，也超出這個 Claude Code 專案一貫「不碰 SAP 標準物件、不做繞過管控的操作」的原則。

真正該走的路：這個檢查是 SAP 官方故意擋下的 (連程式碼註解都寫明「尚未對外釋出」)，代表這套系統目前的 Support Package 等級還沒到 SAP 決定正式開放的門檻。合理的下一步是：① 請 Basis 團隊查證目前 `SAP_BASIS` / `S4CORE` 的 Support Package 版本，② 到 SAP Support Portal 用 `CL_CSP_MD_METADATA_FACTORY` 或這則錯誤訊息文字查詢對應的 OSS Note，確認哪個 SP 起會正式開放、有沒有需要額外申請的開關；Claude 這邊沒有 S-user 帳號查不到，這件事需要使用者或 Basis 窗口跟進。在那之前，這系統上想要「真的能寫入」的 RAP CUD，只能走 Unmanaged (已完整驗證能跑)，這也是這門課從這一課起 Managed / Unmanaged 並教的原因。

意外收穫：這也讓我們搞清楚了一個誤判——`.claude/rules/sap-adt-mcp.md` 第 42 節原本記錄「EML 沒辦法用 `programrun` 無頭驗證」，但這次用 Unmanaged BDEF 的 EML 測試完全無頭執行成功，證實

問題從來不是 **EML** 這個語言機制，而是呼叫到 Managed Runtime 時觸發的致命 Dump，透過 RFC Bridge 傳回時卡住了連線。這個推測已經更正。

學習目標

- 能寫出這系統適用的 Managed BDEF 語法 (`managed;`、`persistent table`、`lock master`、`etag < 欄位>` 不含 `master`)，知道這是知識儲備、銜接未來 ABAP Cloud RAP 課程用
- 能寫出這系統適用的 Unmanaged BDEF 語法 (`implementation unmanaged in class ... unique;`，無 `persistent table`)，知道跟 Managed 的語法差異
- 能寫出 Unmanaged 實作類別的基本結構：繼承 `cl_abap_behavior_handler`、`FOR LOCK/FOR MODIFY/FOR READ` 方法綁定語法
- 能講出 Managed / Unmanaged 的完整差異對照 (程式碼份量、適用情境、這系統的可執行性)
- 知道這系統 Managed Runtime CUD 無法執行的具體原因 (`CL_CSP_MD_METADATA_FACTORY` 白名單機制，SAP 官方尚未對客戶套件開放，推測跟 S/4HANA 1909 版本有關)
- 能寫出基礎的 EML 語法 (`MODIFY ENTITIES OF ... CREATE`、`COMMIT ENTITIES`)，知道 `CREATE/UPDATE` 用 `WITH VALUE`、`DELETE` 用 `FROM VALUE`
- 能在 Eclipse ADT 完整走過一次「對著 CDS View 右鍵 → New Behavior Definition → 選 Implementation Type → Next → Transport → Finish」的精靈流程，知道 BDEF 名稱固定跟 CDS View 同名不能改；Unmanaged 情境下能用 `Ctrl+1` 快速修正一鍵生成實作類別骨架

物件清單

物件	名稱	型別	可執行性
Managed Behavior Definition	<code>ZI_RAP02_TASK</code> (沿用 rap02 的 CDS View)	<code>BDEF/BDO</code>	語法正確，CUD 無法執行
Managed EML 驗證程式	<code>ZR_RAP03_DEMO</code>	<code>PROG/P</code>	語法正確，執行會 Dump
Unmanaged 測試表格	<code>ZRAP03_UMTEST</code>	<code>TABL/DT</code>	—
Unmanaged CDS View	<code>ZI_RAP03_UMTEST</code>	<code>DDLS/DF</code>	—
Unmanaged Behavior Definition	<code>ZI_RAP03_UMTEST</code>	<code>BDEF/BDO</code>	✓
Unmanaged 實作類別	<code>ZBP_I_RAP03_UM4</code>	<code>CLAS/OC</code>	✓
Unmanaged EML 驗證程式	<code>ZR_RAP03_UMTEST</code>	<code>PROG/P</code>	✓ 已驗證成功

全部物件都在 `$TMP` 套件，`sap_inactive_objects` 確認 0 筆殘留。(「Eclipse ADT 建立 Behavior Definition」練習用的物件由你自己命名，不算進這份正式物件清單，跟 rap02 的練習物件是同樣的處理方式。)

驗證方式

- Managed** 部分：`checkruns API` / `sap_inactive_objects` 確認語法正確、成功啟用即可——不要嘗試執行 `ZR_RAP03_DEMO`，不管在 SAP GUI 或 `programrun`，一律會遇到 Part C 說明的 Dump，這是預期中的、已經查清楚原因的結果，不用重複驗證

2. **Unmanaged** 部分：已用 `programrun` 完整驗證成功，輸出見上方 Part B，資料庫比對也確認寫入成功

思考題

1. `field ( readonly ) created_at, created_by` (Managed 版本) 現在阻止了使用者自己填這兩個欄位，但也沒有任何機制會自動填值——rap05 要怎麼補上「自動填值」這個邏輯？(提示：想想 Determination 的 `on save/on modify` 時機；但別忘了 Managed CUD 這系統執行不了，這一題純粹是語法層面的思考)
2. Unmanaged 版本的 `create` 方法，如果同時要寫入兩張表 (例如 Header+Item)，實作邏輯要怎麼設計？跟 Managed 版本只要在 BDEF 宣告 Composition 就能自動處理，複雜度差在哪裡？
3. 如果之後真的解除了這系統 Managed Runtime 的白名單限制 (例如 Support Package 升級後)，Part A 寫的 Managed BDEF 語法要改多少才能真正動起來？(提示：語法本身可能完全不用改，只是「能不能執行」這件事跟語法無關)

答案

Managed : `zi_rap02_task.bdef.abap`、`zr_rap03_demo.prog.abap`。 **Unmanaged** : `zrap03_umtest.tabl.abap`、`zi_rap03_umtest.ddls.abap`、`zi_rap03_umtest.bdef.abap`、`zbp_i_rap03_um4.clas.abap` (Global 類別本體)、`zbp_i_rap03_um4.clas.locals_imp.abap` (Local Handler 實作)、`zr_rap03_umtest.prog.abap` (EML 驗證程式，已驗證執行成功)。